RandomRingtone v0.7.32 — Volledige Flow Analyse

Gegenereerd: 2026-04-02 | Branch: spotify | Build 53 "Prince" / "Uptown"

Inhoudsopgave

1. Opstartsequentie
2. Spotify Tab — Zoeken & Downloaden
3. Editor — Waveform & Trimmen
4. Library Tab — Bibliotheek
5. Playlists Tab — Playlistbeheer
6. Overzicht Tab
7. Backup Tab
8. Instellingen Tab
9. Achtergrondcomponenten
10. Systeemtoegang Matrix
11. Permissie Matrix
12. Datamodel

---

1. Opstartsequentie

Fase 1: Android System → Activity (synchrone startup)

```
Android Launcher tap
  └→ ActivityManager start MainActivity
       └→ onCreate() [MainActivity.kt:26]
            ├→ enableEdgeToEdge()              — window insets configureren (visueel)
            └→ setContent { ... }              — Compose runtime starten
```

Systeemtoegang: ActivityManager, WindowManager **Duur:** ~50-100ms

Fase 2: Compose compositie (synchrone UI-opbouw)

```
setContent
  └→ RandomRingtoneTheme [Theme.kt:10]
       ├→ isSystemInDarkTheme()               — systeem dark mode lezen
       ├→ dynamicColorScheme(context)          — wallpaper-kleuren ophalen (Android 12+)
       └→ MaterialTheme { RandomRingtoneApp() }
```

```
RandomRingtoneApp() [MainActivity.kt:39]
  ├→ rememberNavController()                  — NavHostController aanmaken
  ├→ mutableIntStateOf(0)                     — selectedTab = 0 (Zoeken)
  ├→ AppRingtoneManager(context)              — CONSTRUCTOR:
  │    ├→ OkHttpClient()                      — HTTP client (connection pool, thread pool)
  │    └→ StorageManager(context)             — CONSTRUCTOR (niks — DataStore lazy)
  ├→ RingtoneDatabase.getInstance()           — Room DB CONSTRUCTOR:
  │    └→ Room.databaseBuilder().build()      — DB object aanmaken, NIET geopend
  │       .fallbackToDestructiveMigration()   — DESTRUCTIEF bij versieconflict!
  ├→ BackupManager(context)                   — CONSTRUCTOR:
  │    └→ Json { prettyPrint, ignoreUnknownKeys }
  └→ SnackbarHostState()
```

Op dit punt: Alle objecten bestaan. Geen DB-verbinding, geen disk I/O, geen netwerk.

Fase 3: Scaffold renderen (synchrone UI)

```
Scaffold [MainActivity.kt:70]
  ├→ TopAppBar — "RandomRingtone v0.7.32"
  ├→ NavigationBar — 6 tabs renderen (Zoeken geselecteerd)
  └→ NavHost(startDestination = "spotify")
       └→ SpotifyScreen() compositie start
            ├→ ~20 mutableStateOf() calls     — UI state initialiseren
            └→ (Compose frame gerenderd → gebruiker ziet de app)
```

Fase 4: LaunchedEffects (asynchrone coroutines, na eerste frame)

4a. Auto-restore check `[MainActivity.kt:49]`

```
LaunchedEffect(Unit)
  └→ backupManager.autoRestoreFromLocal(db, storage) — Dispatchers.IO
       ├→ db.savedTrackDao().getAll()         ← EERSTE DB-OPERATIE: Room opent DB
       │    └→ SQLite: OPEN "randomringtone.db"
```

```
        │     └→ SQLite: CREATE TABLE IF NOT EXISTS × 3
        │     └→ SQLite: SELECT * FROM saved_tracks
        ├→ db.playlistDao().getAll()
        │     └→ SQLite: SELECT * FROM playlists
        ├→ IF beide leeg EN filesDir/auto_backup/ bestaat:
        │     ├→ File.readText("saved_tracks.json")
        │     ├→ Json.decodeFromString() → db.savedTrackDao().insertAll()
        │     ├→ File.readText("playlists.json")
        │     ├→ db.playlistDao().insert() per playlist
        │     ├→ File.readText("playlist_tracks.json")
        │     ├→ db.playlistTrackDao().insert() per koppeling
        │     ├→ File.readText("settings.json")
        │     └→ storage.setDownloadDir/setRingtoneDir — DataStore writes
        └→ IF restored → snackbar "Data hersteld vanuit lokale backup"
```

Systeemtoegang: SQLite (Room), FileSystem (filesDir), DataStore

4b. Auto-backup `[MainActivity.kt:57]`

```
LaunchedEffect(selectedTab = 0)
  └→ backupManager.autoBackupToLocal(db, storage) — Dispatchers.IO
       ├→ db.savedTrackDao().getAll()             — SQLite SELECT
       ├→ db.playlistDao().getAll()               — SQLite SELECT
       ├→ db.playlistTrackDao().getAll()           — SQLite SELECT
       ├→ Json.encodeToString() × 3               — serialize
       ├→ File.writeText() × 3                    —
saved_tracks/playlists/playlist_tracks.json
       ├→ storage.getDownloadDir()                 — DataStore read
       ├→ storage.getRingtoneDir()                 — DataStore read
       ├→ storage.getSpotifyConverter()            — DataStore read
       ├→ storage.getBackupUri()                   — DataStore read
       ├→ File.writeText("settings.json")
       └→ File.writeText("backup_meta.json")
```

Trigger: Elke tab-wissel (niet geoptimaliseerd — draait ook als er niets gewijzigd is).

4c. SpotifyScreen LaunchedEffects `[SpotifyScreen.kt:103-203]`

```
LaunchedEffect(Unit) [r103]
  ├→ storage.getSpotifyConverter()            — DataStore read
  ├→ SpotifyConverter.findById()              — in-memory lookup
  └→ storage.isDirectApiEnabled()             — DataStore read

DisposableEffect(Unit) [r112] — DownloadManager receiver
  └→ context.registerReceiver(
       BroadcastReceiver(ACTION_DOWNLOAD_COMPLETE), RECEIVER_EXPORTED
     )
  — Blijft actief zolang SpotifyScreen in compositie is
  — onDispose: unregisterReceiver()

DisposableEffect(Unit) [r184] — Clipboard listener
  └→ clipboardManager.addPrimaryClipChangedListener()
  — Monitort klembord LIVE voor Spotify URLs
  — onDispose: removePrimaryClipChangedListener()

LaunchedEffect(currentUrl) [r174]
  └→ SPOTIFY_TRACK_REGEX.find(currentUrl)
  — Bij elke URL-wijziging: check of het een Spotify track URL is
```

Fase 5: Stilstand — wacht op gebruikersactie

```
Na ~200-500ms is ALLES klaar:

ACTIEF (wachtend op events):
  ├→ Compose UI thread                      — user input (taps, swipes)
  ├→ BroadcastReceiver                      — DOWNLOAD_COMPLETE intents
  ├→ ClipboardManager listener              — klembord-wijzigingen
  ├→ Spotify WebView                        — laadt open.spotify.com (eigen threads)
  └→ OkHttpClient thread pool               — idle

NIET ACTIEF (geregistreerd in Manifest, gewekt door Android):
  ├→ NotificationService                    — alleen actief als gebruiker het aanzet
  ├→ CallStateReceiver                      — gewekt door PHONE_STATE broadcasts
  └→ RingtoneWorker (WorkManager)           — gewekt door schema
```

---

2. Spotify Tab — Zoeken & Downloaden

2a. Spotify WebView browsen

```
TRIGGER: Gebruiker navigeert in Spotify WebView
INPUT:   URL navigatie op open.spotify.com
FLOW:
  WebViewClient.onPageStarted(url)
    └→ isLoading = true; currentUrl = url
  WebViewClient.onPageFinished(url)
    └→ isLoading = false; canGoBack = webView.canGoBack()
  LaunchedEffect(currentUrl) [r174]
    └→ SPOTIFY_TRACK_REGEX check → detectedTrackUrl = url (of null)
OUTPUT:  FAB "Download MP3" verschijnt/verdwijnt
SYSTEEM: WebView engine (Chromium), CookieManager
```

2b. Klembord-detectie (Spotify "Delen → Link kopiëren")

```
TRIGGER: Klembord wijzigt (gebruiker kopieert Spotify link buiten de app)
INPUT:   Klembordtekst
FLOW:
  ClipboardManager.OnPrimaryClipChangedListener [r185]
    ├→ clipboardManager.primaryClip.getItemAt(0).text
    ├→ SPOTIFY_TRACK_REGEX match check
    └→ IF match EN niet al verwerkt → detectedTrackUrl = clipUrl
OUTPUT:  FAB "Download MP3" verschijnt
SYSTEEM: ClipboardManager
```

2c. Directe API download (SpotMate)

```
TRIGGER: Gebruiker tikt FAB + useDirectApi=true
INPUT:   Spotify track URL, page title
FLOW:
  1. Artiest/titel extraheren uit page title via regex [r283]
  2. spotMateClient.fetchTrackInfo(trackUrl) [r302]
     ├→ GET spotmate.online/en1                - CSRF token + cookies
     └→ POST spotmate.online/getTrackData      - track metadata
  3. showConfirmTrackDialog = true              - vergelijk SpotMate vs Spotify
  4. Gebruiker bevestigt → spotMateClient.downloadTrack() [r470]
     ├→ CSRF token ophalen                     - GET /en1
     ├→ Track metadata ophalen                 - POST /getTrackData
     ├→ Duplicate check: File.exists()
     ├→ Conversie starten                      - POST /convert
     ├→ IF task_id → pollTask()                - GET /tasks/{id} (max 40×, 4.5s interval)
     └→ Download MP3                           - GET download URL → File write
  5. IF fileExists → showOverwriteDialog
     ELSE → showActionsDialog
OUTPUT:  MP3 bestand in downloadDir (spotify_mp3_<track>-<artiest>.mp3)
SYSTEEM: OkHttpClient → spotmate.online API, FileSystem
NETWERK: GET/POST spotmate.online (4+ calls), MP3 download
```

2d. WebView Converter download

```
TRIGGER: Gebruiker tikt FAB + useDirectApi=false
INPUT:   Spotify track URL
FLOW:
  1. URL naar klembord kopiëren [r318]
  2. showConverter = true → ConverterDialog opent
  3. Converter WebView laadt converter-site (bijv. spotifydown.com)
  4. Gebruiker plakt URL, start conversie, klikt download
  5. WebView.setDownloadListener → onDownloadStart [r374]
     ├→ DownloadManager.Request aanmaken
     ├→ setDestinationInExternalPublicDir(Downloads/)
     └→ dm.enqueue(request)
  6. BroadcastReceiver ontvangt ACTION_DOWNLOAD_COMPLETE [r114]
     ├→ DownloadManager.query() → lokaal pad ophalen
     ├→ File.copyTo() → spotify_mp3_<naam>.mp3 in downloadDir
     ├→ IF destFile.exists() → showOverwriteDialog
     └→ ELSE → showActionsDialog
OUTPUT:  MP3 bestand in downloadDir
SYSTEEM: WebView, DownloadManager, BroadcastReceiver, FileSystem
NETWERK: Converter-site, MP3 download via DownloadManager
```

2e. Acties na download

```
TRIGGER: showActionsDialog = true (na succesvolle download)
INPUT:   lastDownloadedFile (File)
OPTIES:
  A. "Openen in editor" [r610]
     └→ Navigate naar EditorScreen via savedStateHandle
  B. "Opslaan in bibliotheek" [r623]
     └→ db.savedTrackDao().insert(SavedTrack) — SQLite INSERT
        trackId = file.name.hashCode().toLong()
        playlistName = "_spotify"
OUTPUT:  Track in DB of navigatie naar editor
SYSTEEM: Room (SQLite), NavController
```

---

3. Editor — Waveform & Trimmen

3a. Editor laden

```
TRIGGER: Navigatie vanuit SpotifyScreen of LibraryScreen
INPUT:   trackTitle, trackArtist, audioFile, deezerTrackId, previewUrl
FLOW:
  LaunchedEffect(audioFile) [EditorScreen.kt:58]
    └→ AudioDecoder.extractWaveform(audioFile) — Dispatchers.Default
         ├→ MediaExtractor.setDataSource(file)
         ├→ MediaCodec.createDecoderByType("audio/mpeg")
         ├→ Feed input buffers → Decode → Read PCM output
         ├→ RMS amplitude berekening per blok
         └→ Downsample naar 500 punten (genormaliseerd 0.0-1.0)
  LaunchedEffect(Unit)
    └→ db.playlistDao().getAll() — bestaande playlists ophalen
  endFraction = min(20s, durationMs) / durationMs  — default selectie 20 sec
OUTPUT:  WaveformData (amplitudes, durationMs, sampleRate)
SYSTEEM: MediaCodec, MediaExtractor, CPU (PCM decoding)
```

3b. Preview afspelen

```
TRIGGER: Gebruiker tikt "Preview"
INPUT:   audioFile, startMs, endMs
FLOW:
  AudioPlayer.playSelection(file, startMs, endMs) [AudioPlayer.kt:24]
    ├→ MediaPlayer().setDataSource(file)
    ├→ prepare() → seekTo(startMs) → start()
    └→ CoroutineScope(Main).launch { delay(duration); stop() }
OUTPUT:  Audio afspelen via speaker/headphones
SYSTEEM: MediaPlayer
```

3c. Opslaan (trim + DB + optioneel ringtone)

```
TRIGGER: Gebruiker tikt "Opslaan" → vult dialoog in → bevestigt
INPUT:   ringtoneName, playlist keuze, setAsMainRingtone toggle
FLOW:
  1. AudioTrimmer.trim(audioFile, trimmedFile, startMs, endMs) [AudioTrimmer.kt:29]
     ├→ MediaExtractor.setDataSource(input)
     ├→ seekTo(startUs, SEEK_TO_CLOSEST_SYNC)
     ├→ IF MP3: direct frame copy → FileOutputStream
     │    (zelfstandige MP3 frames, geen container nodig)
     └→ IF AAC: MediaMuxer → MPEG4 container
  2. db.savedTrackDao().insert(SavedTrack) [EditorScreen.kt:322]
     — localPath = trimmedFile.absolutePath
  3. Playlist koppeling:
     IF createNew → db.playlistDao().insert(Playlist)
     db.playlistTrackDao().insert(PlaylistTrack) [r339]
  4. IF setAsMainRingtone:
     ringtoneManager.setAsRingtone(deezerTrack, trimmedFile) [RingtoneManager.kt:62]
       ├→ Settings.System.canWrite() check
       ├→ addToMediaStore() [r77]:
       │    ├→ ContentValues: DISPLAY_NAME, IS_RINGTONE=true
       │    ├→ ContentResolver.delete() — verwijder bestaand item
       │    ├→ ContentResolver.insert() → URI
       │    └→ ContentResolver.openOutputStream() → file copy
```

```
        └→ RingtoneManager.setActualDefaultRingtoneUri(TYPE_RINGTONE, uri)
OUTPUT:  Getrimd MP3 bestand, DB records, optioneel systeem ringtone
SYSTEEM: MediaExtractor, FileSystem, Room, MediaStore, RingtoneManager, Settings.System
```

---

4. Library Tab — Bibliotheek

4a. Laden

```
TRIGGER: Navigatie naar Library tab
INPUT:   —
FLOW:
  LaunchedEffect(Unit) → refresh() [LibraryScreen.kt:188]
    └→ db.savedTrackDao().getAll() — SQLite SELECT
       .filter { localPath bevat "RandomRingtone" OF starts with
"spotify_mp3_/youtube_mp3_/ringtone_/download_" }
       .map { → LibraryItem(file, trackId, title, artist, size) }
       SPLIT:
         ringtones = items waar file.name starts with "ringtone_"
         downloads = alle overige
OUTPUT:  Twee lijsten: downloads en ringtones
SYSTEEM: Room (SQLite), FileSystem (File.exists, File.length)
```

4b. Scan (bestanden synchroniseren met DB)

```
TRIGGER: Gebruiker tikt "Scan" knop
INPUT:   —
PRECONDITIES: READ_MEDIA_AUDIO permissie (Android 13+)
FLOW:
  IF geen permissie → requestPermission(READ_MEDIA_AUDIO) → wacht
  doScan() [r112]:
    storage.scanExistingFiles() [StorageManager.kt:261]
      ├→ scanDir(ringtoneDir, "ringtone")     — File.listFiles() + parseFileName()
      ├→ scanDir(downloadDir, "download")      — File.listFiles() + parseFileName()
      ├→ scanDir(systemDownloadDir, "system")   — File.listFiles() + parseFileName()
      ├→ parseFileName() patronen:
      │    download_<id>.mp3                    → trackId = id
      │    ringtone_<id>.mp3                    → trackId = id
      │    ringtone_<id>_<playlist>.mp3         → trackId = id, playlist
      │    spotify_mp3_<track>-<artiest>.mp3    → trackId = name.hashCode()
      │    overig                               → trackId = name.hashCode()
      └→ IF results.isEmpty() → MediaStore fallback [r319]:
           ContentResolver.query(Audio.Media)
             WHERE DATA LIKE "%RandomRingtone%" OR DISPLAY_NAME LIKE "spotify_mp3_%"
    Per gevonden bestand:
      IF niet in DB → db.savedTrackDao().insert()
      IF in DB maar localPath ongeldig → update localPath
    refresh() — UI herladen
OUTPUT:  Nieuwe tracks in DB, snackbar "X nieuw van Y bestanden"
SYSTEEM: FileSystem, MediaStore (fallback), Room, READ_MEDIA_AUDIO permissie
```

4c. Verwijderen

```
TRIGGER: Gebruiker tikt delete icon → kiest optie
INPUT:   LibraryItem
OPTIES:
  A. "Uit bibliotheek verwijderen"
     └→ db.savedTrackDao().delete(track)      — SQLite DELETE (DB only)
  B. "Verwijder van schijf (permanent)"
     ├→ db.savedTrackDao().delete(track)      — SQLite DELETE
     └→ item.file.delete()                    — FileSystem DELETE
```

```
OUTPUT:  Track uit DB, optioneel bestand van schijf
SYSTEEM: Room, FileSystem
```

4d. Trim (navigatie naar editor)

```
TRIGGER: Gebruiker tikt scissors icon op een download
INPUT:   LibraryItem (title, artist, file, trackId)
FLOW:
  onOpenEditor callback → savedStateHandle → navigate("editor")
OUTPUT:  Navigatie naar EditorScreen
SYSTEEM: NavController, SavedStateHandle
```

---

5. Playlists Tab — Playlistbeheer

5a. Laden

```
TRIGGER: Navigatie naar Playlists tab
INPUT:   —
FLOW:
  LaunchedEffect(Unit) → refresh() [PlaylistManagerScreen.kt:72]
    ├→ db.playlistDao().getAll()               — SQLite SELECT
    └→ per playlist: db.playlistTrackDao().getTrackCount()
OUTPUT:  Playlist-lijst met track counts
SYSTEEM: Room
```

5b. Playlist aanmaken/bewerken

```
TRIGGER: Gebruiker tikt "Nieuw" of "Bewerk"
INPUT:   Naam, channel, mode, schedule, scope (globaal/contact)
PRECONDITIES:
  Per-contact scope vereist READ_CONTACTS + WRITE_CONTACTS
FLOW:
  PlaylistEditDialog [r368]:
    IF per-contact:
      contactsPermissionLauncher.launch([READ_CONTACTS, WRITE_CONTACTS])
      contactsRepo.getContacts() [ContactsRepository.kt:22]
        └→ ContentResolver.query(Contacts.CONTENT_URI)
           WHERE HAS_PHONE_NUMBER = 1
           → List<ContactInfo>(uri, name, photoUri)
    Gebruiker selecteert contact + configuratie
  onSave:
    IF edit → db.playlistDao().update()
    IF new  → db.playlistDao().insert() → returns playlistId
    IF isActive:
      conflictResolver.enforceOneActivePerChannelScope(id) [ConflictResolver.kt:101]
        └→ db.playlistDao().deactivateOtherGlobal/ForContact()
    RingtoneWorker.scheduleAll(context) [RingtoneWorker.kt:33]
      ├→ wm.cancelUniqueWork() × 3
      └→ wm.enqueueUniquePeriodicWork() × 3 (hourly, daily, weekly)
    IF CALL + isActive:
      TrackResolver.applyCallPlaylist(playlist) [TrackResolver.kt:150]
        ├→ resolveForPlaylist() → track + file
        ├→ IF per-contact:
        │    ├→ ringtoneManager.addToMediaStorePublic() → URI
        │    └→ ContactsRepository.setContactRingtone(contactUri, uri)
        │         └→ ContentResolver.update(Contacts, CUSTOM_RINGTONE)
        └→ IF globaal:
             └→ ringtoneManager.setAsRingtone()
                └→ addToMediaStore() + setActualDefaultRingtoneUri()
OUTPUT:  Playlist in DB, workers gepland, ringtone optioneel actief
SYSTEEM: Room, WorkManager, ContactsContract, MediaStore, RingtoneManager
```

5c. Playlist activeren/deactiveren (toggle)

```
TRIGGER: Gebruiker schakelt switch
INPUT:   Playlist
FLOW:
  db.playlistDao().update(isActive toggled)
```

```
  IF nu actief:
    conflictResolver.enforceOneActivePerChannelScope()
    IF CALL + per-contact:
      IF geen WRITE_CONTACTS → requestPermission()
      ELSE → TrackResolver.applyCallPlaylist()
    IF CALL + globaal:
      TrackResolver.applyCallPlaylist()
    RingtoneWorker.scheduleAll()
OUTPUT:  Playlist (de)geactiveerd, conflicten opgelost, ringtone toegepast
SYSTEEM: Room, WorkManager, ContactsContract, MediaStore
```

5d. Tracks toevoegen aan playlist

```
TRIGGER: Gebruiker tikt "Tracks" → AddTracksDialog
INPUT:   Playlist, beschikbare ringtones uit DB
FLOW:
  LaunchedEffect(playlist.id) [r619]:
    ├→ db.savedTrackDao().getAll() + zelfde filter als Library
    └→ db.playlistTrackDao().getTracksForPlaylist()
  Gebruiker selecteert/deselecteert tracks
  Opslaan:
    toRemove = currentTrackIds - selectedIds → db.playlistTrackDao().remove()
    toAdd = selectedIds - currentTrackIds → db.playlistTrackDao().insert()
OUTPUT:  PlaylistTrack koppelingen bijgewerkt
SYSTEEM: Room
```

5e. Playlist exporteren

```
TRIGGER: Gebruiker tikt export icon
INPUT:   Playlist
FLOW:
  backupManager.exportPlaylist(playlistId, db) [BackupManager.kt:304]
    ├→ db.playlistDao().getById()
    ├→ db.playlistTrackDao().getAll().filter { playlistId }
    ├→ db.savedTrackDao().getAll().filter { trackIds }
    └→ Json.encodeToString(PlaylistExport)
  exportLauncher.launch("playlist_<naam>.json") — SAF file create
  backupManager.writePlaylistExport(uri, json)
    └→ ContentResolver.openOutputStream(uri) → write
OUTPUT:  JSON bestand op door gebruiker gekozen locatie
SYSTEEM: Room, SAF (Storage Access Framework)
```

5f. Playlist importeren

```
TRIGGER: Gebruiker tikt "Import"
INPUT:   JSON bestand via SAF picker
FLOW:
  importLauncher.launch(["application/json"]) — SAF file pick
  backupManager.importPlaylist(uri, db) [BackupManager.kt:346]
    ├→ ContentResolver.openInputStream(uri) → readText()
    ├→ Json.decodeFromString<PlaylistExport>()
    ├→ db.savedTrackDao().insertAll(tracks) — REPLACE bij conflict
    ├→ Unieke playlist naam genereren (suffix " (1)" etc.)
    ├→ db.playlistDao().insert(Playlist(isActive=false))
    └→ db.playlistTrackDao().insert() per track
```

```
OUTPUT:  Playlist + tracks in DB (altijd inactief bij import)
SYSTEEM: SAF, Room
```

5g. Playlist verwijderen

```
TRIGGER: Gebruiker tikt delete icon
INPUT:   Playlist
FLOW:
  db.playlistTrackDao().removeAll(playlistId)   — DELETE koppelingen
  db.playlistDao().delete(playlist)              — DELETE playlist
  — CASCADE: playlist_tracks worden ook verwijderd door ForeignKey
OUTPUT:  Playlist + koppelingen verwijderd uit DB
SYSTEEM: Room (SQLite CASCADE)
```

---

6. Overzicht Tab

6a. Laden

```
TRIGGER: Navigatie naar Overzicht tab
INPUT:   —
FLOW:
  LaunchedEffect(Unit) → refresh() [OverviewScreen.kt:27]
    conflictResolver.getActiveSettings() [ConflictResolver.kt:37]
      ├→ db.playlistDao().getActive()                 — alle actieve playlists
      └→ per playlist:
           db.playlistTrackDao().getTracksForPlaylist()
           → ActiveSetting(channel, source, name, track, contact, schedule, mode, count)
    Per kanaal:
      conflictResolver.checkConflicts(channel, null) [r78]
        └→ db.playlistDao().getActive().filter { channel }
           IF >1 per kanaal+scope → conflictwaarschuwing
OUTPUT:  Lijst actieve instellingen per kanaal + conflictwaarschuwingen
SYSTEEM: Room
```

Geen user-acties — dit is een read-only overzichtsscherm.

---

7. Backup Tab

7a. Laden

```
TRIGGER: Navigatie naar Backup tab
INPUT:   —
FLOW:
  LaunchedEffect(Unit) [BackupScreen.kt:51]:
    ├→ storage.getBackupUri()                  — DataStore read
    └→ IF backupUri → backupManager.readBackupInfo() → BackupMeta
  LaunchedEffect(selectedProvider) [r59]:
    IF ICT_HORSE:
      ictHorseClient.getStatus() [IctHorseBackupClient.kt:42]
        └→ GET icthorse.nl/randomringtone/backup_api.php?action=status
           Headers: X-Api-Key, X-Device-Id (ANDROID_ID)
    ELSE:
      backupManager.readBackupInfo(backupUri)
        └→ SAF → DocumentFile → "RandomRingtone_Backup/backup_meta.json"
OUTPUT:  Backup metadata (datum, tracks, playlists, bestanden)
SYSTEEM: DataStore, OkHttpClient of SAF, Settings.Secure.ANDROID_ID
```

7b. SAF map kiezen (GDrive/Dropbox/OneDrive)

```
TRIGGER: Gebruiker tikt "Kies map"
INPUT:   —
FLOW:
  directoryPicker.launch(null) → OpenDocumentTree [r75]
  ContentResolver.takePersistableUriPermission(READ + WRITE) [r79]
  storage.setBackupUri(uri) — DataStore write
  backupManager.readBackupInfo(uri) — check bestaande backup
OUTPUT:  Backup locatie opgeslagen in DataStore
SYSTEEM: SAF, DataStore
```

7c. Backup maken (SAF)

```
TRIGGER: Gebruiker tikt "Backup" (GDrive/Dropbox/OneDrive)
INPUT:   backupUri
FLOW:
  backupManager.backup(backupUri, db, storage, onProgress) [BackupManager.kt:88]
    Phase 1: Export DB
      ├→ db.savedTrackDao().getAll()
      ├→ db.playlistDao().getAll()
      └→ db.playlistTrackDao().getAll()
    Phase 2: Write JSON (SAF)
      ├→ DocumentFile.createFile("saved_tracks.json") → write
      ├→ DocumentFile.createFile("playlists.json") → write
      └→ DocumentFile.createFile("playlist_tracks.json") → write
    Phase 3: Copy downloads (SAF)
      └→ Per MP3 in downloadDir → DocumentFile.createFile() → stream copy
    Phase 4: Copy ringtones (SAF)
      └→ Per MP3 in ringtoneDir → DocumentFile.createFile() → stream copy
    Phase 5: Write metadata
      └→ backup_meta.json → write
OUTPUT:  Volledige backup in SAF directory
SYSTEEM: Room, FileSystem, SAF (ContentResolver)
```

7d. Backup maken (iCt Horse)

```
TRIGGER: Gebruiker tikt "Backup" (iCt Horse provider)
INPUT:   —
FLOW:
  ictHorseClient.backup(db, storage, backupManager, onProgress) [IctHorseBackupClient.kt:69]
    Phase 1: POST backup_api.php?action=init
    Phase 2: Export DB → temp JSON files in cacheDir
    Phase 3: Upload JSON files
      └→ Per file: POST ?action=upload (multipart/form-data)
    Phase 4: Upload download MP3s
      └→ Per MP3: POST ?action=upload
    Phase 5: Upload ringtone MP3s
      └→ Per MP3: POST ?action=upload
    Phase 6: POST ?action=complete
    Cleanup: cacheDir temp verwijderen
OUTPUT:  Backup op icthorse.nl server
SYSTEEM: Room, FileSystem, OkHttpClient → icthorse.nl
NETWERK: POST init + N× upload + complete
AUTH:    API key (hardcoded) + ANDROID_ID
```

7e. Restore (SAF)

```
TRIGGER: Gebruiker tikt "Herstellen" → bevestigt
INPUT:   backupUri
FLOW:
  backupManager.restore(backupUri, db, storage, onProgress) [BackupManager.kt:177]
    Phase 1: Validate (backup_meta.json check)
    Phase 2: Read JSON van SAF
    Phase 3: db.clearAllTables() → INSERT alle data
      — localPaths worden herschreven naar huidige ringtoneDir
    Phase 4: Copy MP3 files van SAF → downloadDir + ringtoneDir
OUTPUT:  Volledige DB + bestanden hersteld (VERVANGT alle huidige data)
SYSTEEM: SAF, Room, FileSystem
DESTRUCTIEF: Ja — clearAllTables() wist alle huidige data
```

7f. Restore (iCt Horse)

```
TRIGGER: Gebruiker tikt "Herstellen" → bevestigt (iCt Horse)
INPUT:   —
FLOW:
  ictHorseClient.restore(db, storage, onProgress) [IctHorseBackupClient.kt:166]
    Phase 1: GET ?action=list → bestandslijst
    Phase 2: Download JSON files → cacheDir
    Phase 3: db.clearAllTables() → INSERT alle data
    Phase 4: Download MP3 files → downloadDir + ringtoneDir
OUTPUT:  Volledige DB + bestanden hersteld
SYSTEEM: OkHttpClient, Room, FileSystem
DESTRUCTIEF: Ja
```

---

8. Instellingen Tab

8a. Laden

```
TRIGGER: Navigatie naar Instellingen tab
INPUT:   —
FLOW:
  Permissie checks (synchrone reads) [SettingsScreen.kt:46-53]:
    ├→ Settings.System.canWrite()              — WRITE_SETTINGS
    ├→ isNotificationListenerEnabled()         — Settings.Secure query
    ├→ checkSelfPermission(READ_PHONE_STATE)
    └→ checkSelfPermission(WRITE_CONTACTS)
  LaunchedEffect(Unit) [r314]:
    ├→ storage.getDownloadDir()                — DataStore + File
    ├→ storage.getRingtoneDir()                — DataStore + File
    └→ storage.getDiskUsage()                  — File.listFiles().sumOf { length() }
  LaunchedEffect(Unit) [r538]:
    └→ storage.getSpotifyConverter()            — DataStore read
  LaunchedEffect(Unit) [r601]:
    └→ storage.isDirectApiEnabled()             — DataStore read
  DisposableEffect(lifecycleOwner) [r67]:
    └→ ON_RESUME → hercheck alle permissies (na terugkeer uit Android Settings)
OUTPUT:  Permissie statussen, opslagpaden, converter config
SYSTEEM: Settings.System, Settings.Secure, PackageManager, DataStore, FileSystem
```

8b. WRITE_SETTINGS permissie

```
TRIGGER: Gebruiker tikt "Toestaan"
INPUT:   —
FLOW:
  startActivity(ACTION_MANAGE_WRITE_SETTINGS, "package:<app>")
  — Opent Android systeeminstellingen
  — Bij terugkeer: ON_RESUME → hercheck
OUTPUT:  Permissie status bijgewerkt
SYSTEEM: Android Settings app
```

8c. Notificatie-toegang permissie

```
TRIGGER: Gebruiker tikt "Toestaan"
INPUT:   —
FLOW:
  startActivity(ACTION_NOTIFICATION_LISTENER_SETTINGS)
  — Opent Android Notification Access instellingen
  — Bij terugkeer: ON_RESUME → hercheck via Settings.Secure
OUTPUT:  NotificationService (in)actief
SYSTEEM: Android Settings, NotificationListenerService
```

8d. Opslaglocatie wijzigen

```
TRIGGER: Gebruiker tikt "Wijzig" bij Downloads of Ringtones
INPUT:   —
FLOW:
  OpenDocumentTree → SAF directory picker
  safUriToFilePath(uri) — SAF URI → /storage/emulated/0/...
```

```
  IF succesvol + anders dan huidig → FileMoveDialog:
    OPTIES:
      A. Verplaatsen: storage.moveFilesToNewDir(old, new)
         └→ File.renameTo() of copyTo() + delete()
      B. Kopiëren: storage.copyFilesToNewDir(old, new)
         └→ File.copyTo()
      C. Alleen pad wijzigen
  storage.setDownloadDir/setRingtoneDir(newPath) — DataStore write
OUTPUT:  Nieuw opslagpad, bestanden optioneel verplaatst/gekopieerd
SYSTEEM: SAF, FileSystem, DataStore
```

8e. Spotify Converter wijzigen

```
TRIGGER: Gebruiker selecteert converter uit dropdown
INPUT:   Converter ID
FLOW:
  storage.setSpotifyConverter(converterId) — DataStore write
OUTPUT:  Converter instelling opgeslagen
SYSTEEM: DataStore
```

8f. Directe API toggle

```
TRIGGER: Gebruiker schakelt switch
INPUT:   Boolean
FLOW:
  storage.setDirectApiEnabled(enabled) — DataStore write
OUTPUT:  Download modus opgeslagen (SpotMate API vs WebView converter)
SYSTEEM: DataStore
```

8g. Downloads/Ringtones wissen

```
TRIGGER: Gebruiker tikt "Wissen"
INPUT:   —
FLOW:
  ringtoneManager.clearDownloads() of clearRingtones()
    └→ storage.getDownloadDir/getRingtoneDir().listFiles().forEach { delete() }
OUTPUT:  Alle bestanden in betreffende map verwijderd
SYSTEEM: FileSystem
DESTRUCTIEF: Ja (bestanden permanent verwijderd)
```

---

9. Achtergrondcomponenten

9a. RingtoneWorker (WorkManager — periodiek schema)

```
TRIGGER: WorkManager wekt worker op basis van schema (hourly/daily/weekly)
PRECONDITIES: Netwerk beschikbaar (Constraint)
REGISTRATIE: RingtoneWorker.scheduleAll() bij playlist activering
LIFECYCLE: Onafhankelijk van app — overleeft app-kill, reboot (RECEIVE_BOOT_COMPLETED)

FLOW [RingtoneWorker.kt:78]:
  doWork() — Dispatchers.IO
    ├→ RingtoneDatabase.getInstance()
    ├→ AppRingtoneManager(context)
    ├→ TrackResolver(db, manager, context)
    ├→ Bepaal target schedules op basis van worker tag:
    │    schedule_hourly → HOURLY_1..12
    │    schedule_daily  → DAILY
    │    schedule_weekly → WEEKLY
    ├→ db.playlistDao().getActive()
    │    .filter { RANDOM + matching schedule }
    └→ Per playlist:
         resolver.resolveForPlaylist(playlist) [TrackResolver.kt:32]
           ├→ db.playlistTrackDao().getTracksForPlaylist()
           ├→ IF FIXED → tracks.first()
           │  IF RANDOM → random() (filter lastPlayedTrackId als >1 track)
           └→ resolveTrackFile(track):
                ├→ 1. File(localPath).exists() → return
                ├→ 2. MediaStore fallback → copy naar filesDir/resolved/
                └→ 3. On-the-fly download via Deezer preview URL
         APPLY:
           CALL + per-contact:
             ├→ addToMediaStorePublic() → URI
             └→ ContactsRepository.setContactRingtone(contactUri, uri)
           CALL + globaal:
             └→ ringtoneManager.setAsRingtone()
           NOTIFICATION:
             └→ ringtoneManager.setAsNotification()
           SMS/WHATSAPP:
             └→ (niets — afgehandeld door NotificationService)
         resolver.updateLastPlayed(playlist, trackId)

OUTPUT:  Systeem ringtone/notificatie gewijzigd, contact ringtone gewijzigd
SYSTEEM: WorkManager, Room, FileSystem, MediaStore, RingtoneManager, ContactsContract
NETWERK: Alleen als track niet lokaal beschikbaar (Deezer preview download)
FOUT:    Result.retry() bij exception
```

9b. CallStateReceiver (EVERY_CALL — na elk gesprek)

```
TRIGGER: Android broadcast PHONE_STATE (inkomend/uitgaand gesprek beëindigd)
PRECONDITIES: READ_PHONE_STATE permissie
REGISTRATIE: Manifest-declared receiver (altijd actief)

FLOW [CallStateReceiver.kt:24]:
  onReceive(context, intent)
    ├→ Parse state: IDLE/RINGING/OFFHOOK
    ├→ IF transitie naar IDLE (was OFFHOOK of RINGING):
    │    handleCallEnded(context) — CoroutineScope(IO)
    │      ├→ RingtoneDatabase.getInstance()
```

```
    │      ├→ AppRingtoneManager(context)
    │      ├→ TrackResolver(db, manager, context)
    │      ├→ db.playlistDao().getActive()
    │      │    .filter { CALL + EVERY_CALL }
    │      └→ Per playlist:
    │           resolver.resolveForPlaylist() → (file, track)
    │           IF per-contact:
    │             ├→ addToMediaStorePublic() → URI
    │             └→ ContactsRepository.setContactRingtone()
    │           IF globaal:
    │             └→ ringtoneManager.setAsRingtone()
    │           resolver.updateLastPlayed()
    └→ lastState = currentState (companion object — overleeft calls)

OUTPUT:  Ringtone gewisseld naar volgende track na elk gesprek
SYSTEEM: TelephonyManager, Room, MediaStore, RingtoneManager, ContactsContract
```

9c. NotificationService (SMS/WhatsApp interceptie)

```
TRIGGER: Android stuurt notificatie van WhatsApp, Samsung Messages of Google Messages
PRECONDITIES: Notificatie-toegang ingeschakeld door gebruiker
REGISTRATIE: Manifest service met BIND_NOTIFICATION_LISTENER_SERVICE
LIFECYCLE: Draait als system service zolang notificatie-toegang actief is

FLOW [NotificationService.kt:41]:
  onNotificationPosted(sbn)
    ├→ Bepaal kanaal op basis van package:
    │    com.whatsapp → WHATSAPP
    │    com.samsung.android.messaging → SMS
    │    com.google.android.apps.messaging → SMS
    │    anders → return (niet relevant)
    ├→ extractContactName(sbn, channel)
    │    └→ Notification.EXTRA_TITLE (voor zowel WA als SMS)
    ├→ findPlaylist(contactName, channel) [r89]
    │    ├→ EERST: per-contact match (contactName + channel)
    │    └→ FALLBACK: globale match (channel, contactUri=null)
    ├→ trackResolver.resolveForPlaylist(playlist)
    ├→ playCustomSound(file) [r104]
    │    └→ MediaPlayer (USAGE_NOTIFICATION_RINGTONE)
    │       setDataSource → prepare → start → onCompletion: release
    ├→ trackResolver.updateLastPlayed()
    └→ cancelNotificationSound(sbn) [r123]
         └→ snoozeNotification(key, 1ms) — probeer origineel geluid te dempen

  onCreate() — service initialisatie
    ├→ RingtoneDatabase.getInstance()
    ├→ AppRingtoneManager(this)
    └→ TrackResolver(db, manager, this)

  onDestroy()
    ├→ scope.cancel()
    └→ mediaPlayer.release()

OUTPUT:  Custom geluid afgespeeld, origineel gedempt
SYSTEEM: NotificationListenerService, MediaPlayer, Room
RISICO:  contactName match: "Mama" in contacten vs "Ma" in WhatsApp → mismatch
```

---

10. Systeemtoegang Matrix

Component	SQLite	FileSystem	MediaStore	DataStore	Netwerk	ContactsContract	Settings
Startup	R	R/W	—	R	—	—	—
SpotifyScreen	W	R/W	—	R	spotmate.online	—	—
EditorScreen	R/W	R/W	W	R	—	—	R
LibraryScreen	R/W	R	R (scan)	R	—	—	—
PlaylistManager	R/W	R	W	—	—	R/W	—
OverviewScreen	R	—	—	—	—	—	—
BackupScreen	R	R/W	—	R/W	icthorse.nl	—	R
SettingsScreen	—	R/W	—	R/W	—	—	R
RingtoneWorker	R/W	R/W	R/W	R	(deezer)	W	W
CallStateReceiver	R/W	R/W	R/W	—	(deezer)	W	W
NotificationService	R/W	R	R	—	(deezer)	—	—

R = Read, W = Write, (x) = optioneel/fallback

11. Permissie Matrix

Permissie	Type	Waar gevraagd	Waarvoor nodig
`INTERNET`	Normaal (auto)	—	Deezer API, SpotMate API, iCt Horse backup, WebView
`WRITE_SETTINGS`	Speciaal (user-grant via Settings)	SettingsScreen, LibraryScreen	Systeem ringtone/notificatie instellen
`READ_MEDIA_AUDIO`	Runtime (Android 13+)	LibraryScreen (scan)	MediaStore audio bestanden lezen
`POST_NOTIFICATIONS`	Runtime (Android 13+)	(niet expliciet gevraagd)	Notificaties bij ringtone-wissel
`RECEIVE_BOOT_COMPLETED`	Normaal (auto)	—	WorkManager na reboot herstarten
`READ_CONTACTS`	Runtime	PlaylistEditDialog	Contactenlijst tonen
`WRITE_CONTACTS`	Runtime	PlaylistEditDialog, PlaylistManagerScreen	Per-contact CUSTOM_RINGTONE instellen
`READ_PHONE_STATE`	Runtime	SettingsScreen	EVERY_CALL: gesprekstatus detecteren
`BIND_NOTIFICATION_LISTENER_SERVICE`	Speciaal (user-grant via Settings)	SettingsScreen	SMS/WhatsApp notificaties onderscheppen

Permissie-afhankelijkheid per feature

```
Track zoeken (Spotify WebView)     → INTERNET
Track downloaden (SpotMate)        → INTERNET
Track downloaden (converter)       → INTERNET
Track trimmen (Editor)             → (geen extra)
Scan bibliotheek                   → READ_MEDIA_AUDIO (Android 13+)
Systeem ringtone instellen         → WRITE_SETTINGS
Systeem notificatie instellen      → WRITE_SETTINGS
Per-contact ringtone instellen     → WRITE_CONTACTS + WRITE_SETTINGS
Contacten tonen in playlist-dialoog→ READ_CONTACTS
Ringtone wisselen na elk gesprek   → READ_PHONE_STATE
Custom SMS/WhatsApp geluid         → BIND_NOTIFICATION_LISTENER_SERVICE
Backup naar iCt Horse              → INTERNET
Schema-gebaseerde ringtone wissel  → (alleen INTERNET als track niet lokaal)
```

12. Datamodel

Room Database: `randomringtone.db` **(versie 4)**

```
┌──────────────────────────────────────────┐
│ saved_tracks                             │
├──────────────────────────────────────────┤
│ PK  deezerTrackId  LONG                  │
│     title          TEXT                   │
│     artist         TEXT                   │
│     previewUrl     TEXT                   │
│     localPath      TEXT? (absoluut pad)   │
│     playlistName   TEXT  (legacy veld)    │
└──────────────────────────────────────────┘


┌──────────────────────────────────────────┐
│ playlists                                │
├──────────────────────────────────────────┤
│ PK  id                LONG (autoGenerate)  │
│     name              TEXT                 │
│     channel           ENUM (CALL/NOTIF/SMS/WA)│
│     mode              ENUM (FIXED/RANDOM)  │
│     schedule          ENUM (MANUAL..WEEKLY)│
│     contactUri        TEXT? (null=globaal) │
│     contactName       TEXT?                │
│     isActive          BOOLEAN              │
│     lastPlayedTrackId LONG?                │
└──────────────────────────────────────────┘


┌──────────────────────────────────────────┐
│ playlist_tracks                          │
├──────────────────────────────────────────┤
│ PK  playlistId   LONG  →FK playlists.id  │
│ PK  trackId      LONG  →saved_tracks.id  │
│     sortOrder    INT                     │
│                                          │
│ FK  CASCADE on playlists DELETE          │
└──────────────────────────────────────────┘
```

DataStore: `storage_settings` **(Preferences)**

```
download_path      STRING?   — custom download pad
ringtone_path      STRING?   — custom ringtone pad
spotify_converter  STRING    — converter ID (default: "spotifydown")
backup_uri         STRING?   — SAF backup locatie URI
use_direct_api     BOOLEAN   — SpotMate API vs WebView converter
```

Bestandssysteem

```
{filesDir}/
  ├→ downloads/                    — tijdelijke MP3 downloads
  │    └→ download_<trackId>.mp3
  ├→ resolved/                     — MediaStore fallback kopieën
  │    └→ <originele naam>.mp3
  ├→ auto_backup/                  — lokale auto-backup
  │    ├→ saved_tracks.json
```

```
  │     ├→ playlists.json
  │     ├→ playlist_tracks.json
  │     ├→ settings.json
  │     └→ backup_meta.json
  └→ ringtones/                    — legacy locatie
       └→ *.mp3

{externalFilesDir}/Music/RandomRingtone/Ringtones/
  ├→ ringtone_<trackId>.mp3         — getrimde ringtones
  └→ ringtone_<trackId>_<playlist>.mp3

{downloadDir}/ (configureerbaar)
  ├→ spotify_mp3_<track>-<artiest>.mp3
  └→ download_<trackId>.mp3

{systemDownloads}/                   — /storage/emulated/0/Download/
  └→ (converter downloads landen hier via DownloadManager)

Android MediaStore:
  Ringtones/RandomRingtone/          — geregistreerde ringtones (IS_RINGTONE=true)
  Notifications/RandomRingtone/      — geregistreerde notificaties (IS_NOTIFICATION=true)
```

Externe API's

API	URL	Auth	Gebruikt door
Deezer	api.deezer.com	Geen	DeezerApi (search, track, playlist, artist)
SpotMate	spotmate.online	CSRF + cookies	SpotMateDirectClient (track → MP3)
iCt Horse Backup	icthorse.nl/randomringtone/backup_api.php	API key + ANDROID_ID	IctHorseBackupClient

---

Appendix: OkHttpClient instanties

De app maakt **4 aparte OkHttpClient instanties** aan:

1. `AppRingtoneManager.client` — Deezer preview downloads
2. `DeezerApi.client` — Deezer API calls
3. `SpotMateDirectClient.client` — SpotMate API (30s connect, 60s read)
4. `IctHorseBackupClient.client` — iCt Horse backup (30s connect, 60s read/write)

Elke instantie heeft een eigen connection pool en thread pool. Refactor-kandidaat: gedeelde singleton.